

7. ТЕСТИРОВАНИЕ (TESTING)

В этой главе мы рассмотрим различные способы расширения возможностей тестирования в Rust, а также другие виды тестирования, которые вы, возможно, захотите добавить в свой арсенал. Rust поставляется с рядом встроенных средств тестирования, которые хорошо описаны в книге *The Rust Programming Language* и представлены в первую очередь атрибутом `#[test]` и каталогом `tests/`. Они отлично послужат вам в широком диапазоне приложений и масштабов и часто оказываются всем, что нужно, когда вы только начинаете работу над проектом. Однако по мере развития кодовой базы и усложнения ваших потребностей в тестировании вам, возможно, понадобится выйти за рамки простого навешивания `#[test]` на отдельные функции.

Эта глава разделена на две основные части. Первая часть охватывает механизмы тестирования Rust, такие как стандартная тестовая обвязка (test harness) и условно компилируемый тестовый код. Вторая рассматривает другие способы оценки корректности вашего кода на Rust, такие как бенчмаркинг (benchmarking), линтинг (linting) и фаззинг (fuzzing).

Механизмы тестирования в Rust (Rust Testing Mechanisms)

Следите за обновлениями по адресу <https://github.com/rust-lang/rfcs/pull/2318>.

Чтобы понять различные механизмы тестирования, которые предоставляет Rust, вы должны сначала разобраться, как Rust собирает и запускает тесты. Когда вы выполняете `cargo test --lib`, единственное, что делает Cargo особенного, — это передаёт `rustc` флаг `--test`. Этот флаг указывает `rustc` создать тестовый бинарный файл, который запускает все модульные тесты (unit tests), а не просто скомпилировать библиотеку или бинарный файл крейта. За кулисами у `--test` есть два основных эффекта. Во-первых, он включает `cfg(test)`, чтобы вы могли условно включать тестовый код (подробнее об этом чуть позже). Во-вторых, он заставляет компилятор сгенерировать *тестовую обвязку* (test harness): тщательно сгенерированную

функцию `main`, которая при запуске вызывает каждую функцию `#[test]` в вашей программе.

Тестовая обвязка (The Test Harness)

Компилятор генерирует функцию `main` тестовой обвязки с помощью сочетания процедурных макросов, которые мы подробнее обсудим в главе 8, и небольшой доли магии. По сути, обвязка преобразует каждую функцию, помеченную атрибутом `#[test]`, в *дескриптор* (*descriptor*) теста — это процедурно-макросная часть. Затем она предоставляет сгенерированной функции `main` путь к каждому из дескрипторов — это магическая часть. Дескриптор включает информацию вроде имени теста, любых дополнительных параметров, которые он задаёт (как `#[should_panic]`), и так далее. По своей сути тестовая обвязка перебирает тесты в крейте, запускает их, фиксирует их результаты и печатает результаты. Поэтому она также включает логику для разбора аргументов командной строки (для таких вещей, как `--test-threads=1`), захвата вывода теста, запуска перечисленных тестов параллельно и сбора результатов тестов.

На момент написания разработчики Rust работают над тем, чтобы сделать магическую часть генерации тестовой обвязки публично доступным API, чтобы разработчики могли создавать собственные тестовые обвязки. Эта работа всё ещё находится на экспериментальной стадии, но предложение довольно близко соответствует модели, существующей сегодня. Часть магии, которую нужно проработать, — это как обеспечить доступность функций `#[test]` для сгенерированной функции `main`, даже если они находятся внутри приватных подмодулей.

Интеграционные тесты (integration tests) (тесты в каталоге *tests/*) следуют тому же процессу, что и модульные тесты, с одним исключением: каждый из них компилируется как отдельный крейт, а значит, они могут обращаться только к публичному интерфейсу основного крейта и запускаются против основного крейта, скомпилированного без `#[cfg(test)]`. Тестовая обвязка генерируется для каждого файла в *tests/*. Тестовые обвязки не генерируются для файлов в подкаталогах внутри *tests/*, что позволяет вам иметь общие подмодули для ваших тестов.

ПРИМЕЧАНИЕ Если вы явно хотите тестовую обвязку для файла в подкаталоге, вы можете подключить её, назвав файл *main.rs*.

Rust не требует, чтобы вы использовали тестовую обвязку по умолчанию. Вместо этого вы можете отказаться от неё и реализовать собственную функцию `main`, представляющую исполнителя тестов, задав `harness = false` для конкретного интеграционного теста в *Cargo.toml*, как показано в листинге 7-1. Тогда для запуска теста будет вызвана определённая вами функция `main`.

```
[[test]]
name = "custom"
path = "tests/custom.rs"
harness = false
```

Листинг 7-1: Отказ от стандартной тестовой обвязки

Без тестовой обвязки никакой магии вокруг `#[test]` не происходит. Вместо этого предполагается, что вы напишете собственную функцию `main`, чтобы запустить тестовый код, который хотите выполнить. По сути, вы пишете обычный бинарный файл на Rust, который просто запускается командой `cargo test`. Этот бинарный файл отвечает за обработку всех тех вещей, которые обычно делает обвязка по умолчанию (если вы хотите их поддерживать), таких как флаги командной строки. Свойство `harness` задаётся отдельно для каждого интеграционного теста, поэтому у вас может быть один тестовый файл, использующий стандартную обвязку, и другой, который её не использует.

АРГУМЕНТЫ ДЛЯ ТЕСТОВОЙ ОБВЯЗКИ ПО УМОЛЧАНИЮ (ARGUMENTS TO THE DEFAULT TEST HARNESS)

Тестовая обвязка по умолчанию поддерживает ряд аргументов командной строки для настройки того, как запускаются тесты. Они передаются не напрямую в `cargo test`, а тестовому бинарному файлу, который Cargo компилирует и запускает для вас, когда вы выполняете `cargo test`. Чтобы передать этот набор флагов, поставьте `--` после `cargo test`, а затем укажите аргументы для тестового бинарника. Например, чтобы увидеть справочный текст для тестового бинарника, вы выполнили бы `cargo test -- --help`.

Через эти аргументы командной строки доступен ряд удобных параметров конфигурации. Флаг `--nocapture` отключает захват вывода, который обычно происходит при запуске тестов Rust. Это полезно, если вы хотите наблюдать вывод теста в реальном времени, а не весь сразу после того, как тест завершился неудачей. Вы можете

использовать параметр `--test-threads`, чтобы ограничить количество одновременно выполняемых тестов, что полезно, когда у вас есть тест, который зависает или вызывает `segfault`, и вы хотите выяснить, какой именно, запуская тесты последовательно. Также есть параметр `--skip` для пропуска тестов, соответствующих определённому шаблону, `--ignored` для запуска тестов, которые обычно были бы проигнорированы (например, те, что требуют запуска внешней программы), и `--list`, чтобы просто перечислить все доступные тесты.

Имейте в виду, что все эти аргументы реализованы тестовой обвязкой по умолчанию, так что если вы её отключите (с `harness = false`), вам придётся реализовать нужные вам аргументы самостоятельно в своей функции `main`!

Интеграционные тесты без обвязки главным образом полезны для бенчмарков, как мы увидим позже, но они также пригодятся, когда вы хотите запускать тесты, не вписывающиеся в стандартную модель «одна функция — один тест». Например, вы часто будете видеть тесты без обвязки, используемые с фаззерами, проверщиками моделей (`model checkers`) и тестами, требующими пользовательской глобальной настройки (как под `WebAssembly` или при работе с пользовательскими целевыми платформами).

`#[cfg(test)]`

Когда Rust собирает код для тестирования, он устанавливает флаг конфигурации компилятора `test`, который вы затем можете использовать вместе с условной компиляцией, чтобы иметь код, который компилируется только тогда, когда он специально тестируется. На первый взгляд это может показаться странным: разве вы не хотите тестировать ровно тот же код, который идёт в продакшен? Хотите, но наличие кода, доступного исключительно при тестировании, позволяет вам писать лучшие, более тщательные тесты несколькими способами.

МОКИНГ (MOCKING) Когда вы пишете тесты, вам часто требуется жёсткий контроль над тестируемым кодом, а также над любыми другими типами, с которыми ваш код может взаимодействовать. Например, если вы тестируете сетевой клиент, вы, вероятно, не хотите запускать модульные тесты по реальной сети, а вместо этого хотите напрямую контролировать, какие байты выдаёт «сеть». Или, если вы

тестируете структуру данных, вы хотите, чтобы ваш тест использовал типы, которые позволяют контролировать, что возвращает каждый метод при каждом вызове. Вы также можете захотеть собирать метрики, такие как сколько раз был вызван данный метод или была ли выдана данная последовательность байтов.

Эти «фальшивые» типы и реализации известны как *моки (mocks)*, и они являются ключевой особенностью любого обширного набора модульных тестов. Хотя работу, необходимую для получения такого контроля, часто можно выполнить вручную, приятнее, когда библиотека берёт на себя большинство досадных деталей. Именно здесь в игру вступает автоматизированный мокинг (mocking). Библиотека мокинга будет иметь средства для генерации типов (включая функции) с определёнными свойствами или сигнатурами, а также механизмы для контроля и интроспекции этих сгенерированных элементов во время выполнения теста.

Мокинг в Rust обычно осуществляется через дженерики: пока ваша программа, структура данных, фреймворк или инструмент являются обобщёнными относительно всего, что вы могли бы захотеть замочать (или принимают трейт-объект), вы можете использовать библиотеку мокинга для генерации соответствующих типов, которые подставят эти обобщённые параметры. Затем вы пишете модульные тесты, инстанцируя ваши обобщённые конструкции сгенерированными мок-типами, и вперёд!

В ситуациях, где дженерики неудобны или неуместны, например если вы хотите избежать того, чтобы какой-то конкретный аспект вашего типа был обобщённым для пользователей, вы можете вместо этого инкапсулировать состояние и поведение, которое хотите замочать, в отдельную структуру. Затем вы сгенерировали бы замочанную версию этой структуры и её методов и использовали бы условную компиляцию, чтобы применять либо реальную, либо замочанную реализацию в зависимости от `cfg(test)` или тестовой фичи вроде `cfg(feature = "test_mock_foo")`.

На данный момент в сообществе Rust не появилось ни единственной библиотеки мокинга, ни даже единого подхода к мокингу, который стал бы Единственно Верным Ответом. Самая обширная и тщательная

библиотека мокинга, которую я знаю, — это крейт `mockall`, но он всё ещё находится в активной разработке, и есть множество других претендентов.

API только для тестов (Test-Only APIs)

Во-первых, наличие кода только для тестов позволяет вам предоставлять дополнительные методы, поля и типы вашим (модульным) тестам, чтобы они могли проверять не только то, что публичный API ведёт себя корректно, но и то, что внутреннее состояние правильно. Например, рассмотрим тип `HashMap` из крейта `hashbrown`, который реализует `HashMap` стандартной библиотеки. Тип `HashMap` на самом деле является просто обёрткой вокруг типа `RawTable`, который реализует большую часть логики хеш-таблицы. Предположим, что после выполнения `HashMap::insert` на пустой карте вы хотите проверить, что единственный бакет в карте не пуст, как показано в листинге 7-2.

```
#[test]
fn insert_just_one() {
    let mut m = HashMap::new();
    m.insert(42, ());
    let full = m.table.buckets.iter().filter(Bucket::is_full).count();
    assert_eq!(full, 1);
}
```

Листинг 7-2: Тест, который обращается к недоступному внутреннему состоянию и потому не компилируется

Этот код не скомпилируется в таком виде, потому что, хотя тестовый код может обращаться к приватному полю `table` типа `HashMap`, он не может обратиться к также приватному полю `buckets` типа `RawTable`, поскольку `RawTable` находится в другом модуле. Мы могли бы исправить это, сделав видимость поля `buckets` равной `pub(crate)`, но мы на самом деле не хотим, чтобы `HashMap` мог трогать `buckets` вообще, так как это может случайно повредить внутреннее состояние `RawTable`. Даже сделать `buckets` доступным только для чтения может оказаться проблематичным, так как новый код в `HashMap` мог бы начать зависеть от внутреннего состояния `RawTable`, что усложнит будущие модификации.

Решение — использовать `#[cfg(test)]`. Мы можем добавить метод к `RawTable`, который разрешает доступ к `buckets` только при тестировании, как показано в листинге 7-3, и тем самым избежать «выстрелов себе в ногу» в остальном

коде. Код из листинга 7-2 затем можно обновить, чтобы он вызывал `buckets()` вместо обращения к приватному полю `buckets`.

```
impl RawTable {
    #[cfg(test)]
    pub(crate) fn buckets(&self) -> &[Bucket] {
        &self.buckets
    }
}
```

Листинг 7-3: Использование `#[cfg(test)]`, чтобы сделать внутреннее состояние доступным в контексте тестирования

Учёт для тестовых утверждений (Bookkeeping for Test Assertions)

Второе преимущество наличия кода, который существует только во время тестирования, заключается в том, что вы можете дополнять программу выполнением дополнительного учёта во время выполнения, который затем можно проверять тестами. Например, представьте, что вы пишете собственную версию типа `BufWriter` из стандартной библиотеки. При его тестировании вы хотите убедиться, что `BufWriter` не выполняет системные вызовы без необходимости. Самый очевидный способ сделать это — заставить `BufWriter` отслеживать, сколько раз он вызвал `write` на нижележащем `Write`. Однако в продакшене эта информация не важна, а её отслеживание вносит (незначительные) накладные расходы по производительности и памяти. С `#[cfg(test)]` вы можете сделать так, чтобы учёт происходил только при тестировании, как показано в листинге 7-4.

```
struct BufWriter<T> {
    #[cfg(test)]
    write_through: usize,
    // другие поля...
}

impl<T: Write> Write for BufWriter<T> {
    fn write(&mut self, buf: &[u8]) -> Result<usize> {
        // ...
        if self.full() {
            #[cfg(test)]
            self.write_through += 1;
            let n = self.inner.write(&self.buffer[..])?;
            // ...
        }
    }
}
```


Листинг 7-4: Использование `#[cfg(test)]`, чтобы ограничить учёт контекстом тестирования

Имейте в виду, что `test` устанавливается только для крейта, который компилируется как тест. Для модульных тестов это тестируемый крейт, как вы и ожидали бы. Однако для интеграционных тестов это интеграционный тестовый бинарный файл, компилируемый как тест, — крейт, который вы тестируете, компилируется просто как библиотека и потому не будет иметь установленного `test`.

Доктесты (Doctests)

Фрагменты кода на Rust в комментариях документации автоматически запускаются как тестовые случаи. Их обычно называют *доктестами* (*doctests*). Поскольку доктесты появляются в публичной документации вашего крейта, а пользователи, вероятно, будут подражать тому, что в них содержится, они запускаются как интеграционные тесты. Это означает, что доктесты не имеют доступа к приватным полям и методам, а `test` не устанавливается для кода основного крейта. Каждый доктест компилируется как отдельный крейт и запускается изолированно, ровно как если бы пользователь скопировал и вставил доктест в собственную программу.

За кулисами компилятор выполняет некоторую предобработку доктестов, чтобы сделать их более лаконичными. Самое важное: он автоматически добавляет `fn main` вокруг вашего кода. Это позволяет доктестам сосредоточиться только на важных деталях, которые волнуют пользователя, — например, на тех частях, которые действительно используют типы и методы из вашей библиотеки, — без включения лишнего шаблонного кода.

Вы можете отказаться от этого автоматического оборачивания, определив собственную `fn main` в доктесте. Возможно, вы захотите это сделать, например, если хотите написать асинхронную функцию `main`, используя что-то вроде `#[tokio::main] async fn main`, или если хотите добавить дополнительные модули в доктест.

Чтобы использовать оператор `?` в вашем доктесте, обычно не нужно использовать пользовательскую функцию `main`, поскольку `rustdoc` включает некоторую эвристику для установки возвращаемого типа в `Result<(), impl Debug>`, если ваш код выглядит так, будто использует `?` (например, если он

заканчивается на `Ok(()))`. Если вывод типов с трудом справляется с типом ошибки для функции, вы можете устранить неоднозначность, изменив последнюю строку доктеста так, чтобы она была явно типизирована, вот так: `Ok::<(), T>(()))`.

Доктесты имеют ряд дополнительных возможностей, которые пригодятся, когда вы пишете документацию для более сложных интерфейсов. Первая — это возможность скрывать отдельные строки. Если вы предваряете строку доктеста символом `#`, эта строка включается при компиляции и запуске доктеста, но не включается во фрагмент кода, генерируемый в документации. Это позволяет легко скрывать детали, не важные для текущего примера, такие как реализация трейтов для фиктивных типов или генерация значений. Это также полезно, если вы хотите представить последовательность примеров, не показывая каждый раз один и тот же ведущий код. В листинге 7-5 приведён пример того, как может выглядеть доктест со скрытыми строками.

```
/// Полностью «фробнифицирует» число через ввод-вывод.
///
/// В этом первом примере мы скрываем генерацию значения.
/// ```
/// # let unfrobnified_number = 0;
/// # let already_frobnified = 1;
/// assert!(frobnify(unfrobnified_number).is_ok());
/// assert!(frobnify(already_frobnified).is_err());
/// ```
///
/// Вот пример, который использует ? на нескольких типах
/// и потому должен объявить конкретный тип ошибки,
/// но мы не хотим отвлекать этим пользователя.
/// Мы также скрываем use, который вводит функцию в область видимости.
/// ```
/// # use mylib::frobnify;
/// frobnify("0".parse())?;
/// # Ok::<(), anyhow::Error>(()))
/// ```
///
/// Вы могли бы даже заменить целый блок кода полностью,
/// хотя используйте это _очень_ умеренно:
/// ```
/// # /*
/// let i = ...;
/// # */
/// # let i = 42;
/// frobnify(i)?;
/// ```
fn frobnify(i: usize) -> std::io::Result<()> {
```

Листинг 7-5: Скрытие строк в доктесте с помощью #

ПРИМЕЧАНИЕ Используйте эту возможность с осторожностью; пользователям может быть неприятно, если они скопируют и вставят пример, а он затем не работает из-за необходимых шагов, которые вы скрыли.

Как и в случае с тестовыми функциями, доктесты также поддерживают атрибуты, которые изменяют то, как запускается доктест. Эти атрибуты идут сразу после тройного бэктика, используемого для обозначения блока кода, и несколько атрибутов могут быть разделены запятыми.

Как и для тестовых функций, вы можете указать атрибут `should_panic`, чтобы обозначить, что код в конкретном доктесте при запуске должен паниковать, или `ignore`, чтобы проверять сегмент кода только если `cargo test` запускается с флагом `--ignored`. Вы также можете использовать атрибут `no_run`, чтобы обозначить, что данный доктест должен компилироваться, но не должен запускаться.

Атрибут `compile_fail` сообщает `rustdoc`, что код в примере документации не должен компилироваться. Это указывает пользователю, что конкретное использование невозможно, и служит полезным тестом, напоминающим вам обновить документацию, если соответствующий аспект вашей библиотеки изменится. Вы также можете использовать этот атрибут, чтобы проверить, что определённые статические свойства выполняются для ваших типов. В листинге 7-6 показан пример того, как можно использовать `compile_fail`, чтобы проверить, что данный тип не реализует `Send`, что может быть необходимо для соблюдения гарантий безопасности в небезопасном (`unsafe`) коде.

`struct MyNonSendType(std::rc::Rc<()>);`

```
fn is_send<T: Send>() {} is_send::<MyNonSendType>();
```

Листинг 7-6: Проверка того, что код не компилируется, с помощью `compile_fail`

`compile_fail` — довольно грубый инструмент в том смысле, что он не даёт никакого указания на то, *почему* код не компилируется. Например, если код не компилируется из-за пропущенной точки с запятой, тест `compile_fail` будет выглядеть успешным. По этой причине обычно стоит добавлять атрибут

только после того, как вы убедились, что тест действительно не компилируется с ожидаемой ошибкой. Если вам нужны более точные тесты для ошибок компиляции, например при разработке макросов, обратите внимание на крейт `trybuild`.

Дополнительные инструменты тестирования (Additional Testing Tools)

Тестирование — это намного больше, чем просто запуск тестовых функций и проверка того, что они выдают ожидаемый результат. Тщательный обзор техник, методологий и инструментов тестирования выходит за рамки этой книги, но есть некоторые ключевые, специфичные для Rust части, о которых вам следует знать по мере расширения вашего репертуара тестирования.

Линтинг (Linting)

Вы можете не считать проверки линтера тестами, но в Rust они часто могут ими быть. Линтер Rust `clippy` категоризирует ряд своих линтов как *линты корректности* (*correctness lints*). Эти линты ловят шаблоны кода, которые компилируются, но почти наверняка являются багами. Некоторые примеры: `a = b; b = a`, что не приводит к обмену значениями `a` и `b`; `std::mem::forget(t)`, где `t` — это ссылка; и `for x in y.next()`, что проитерирует только по первому элементу в `y`. Если вы ещё не запускаете `clippy` как часть вашего CI-конвейера, вам, вероятно, стоит это делать.

`Clippy` поставляется с рядом других линтов, которые, хотя обычно полезны, могут оказаться более «мнениевыми», чем вам хотелось бы. Например, линт `type_complexity`, который включён по умолчанию, выдаёт предупреждение, если вы используете особенно громоздкий тип в своей программе, вроде `Rc<Vec<Vec<Box<(u32, u32, u32, u32)>>>>`. Хотя это предупреждение побуждает вас писать код, который легче читать, вы можете найти его слишком педантичным для широкого применения. Если какая-то часть вашего кода ошибочно срабатывает на конкретном линте или вы просто хотите разрешить конкретный его случай, вы можете отключить линт только для этого участка кода с помощью `#[allow(clippy::name_of_lint)]`.

Компилятор Rust также поставляется с собственным набором линтов в форме предупреждений, хотя они обычно больше направлены на написание

идиоматичного кода, чем на проверку корректности. Линты корректности в компиляторе вместо этого просто трактуются как ошибки (взгляните на `rustc -W help` для списка).

ПРИМЕЧАНИЕ Не все предупреждения компилятора включены по умолчанию. Те, что отключены по умолчанию, обычно всё ещё дорабатываются или больше относятся к стилю, чем к содержанию. Хороший пример этого — линт «idiomatic Rust 2018 edition», который можно включить с помощью `#![warn(rust_2018_idioms)]`. Когда этот линт включён, компилятор сообщит вам, если вы не пользуетесь преимуществами изменений, привнесённых редакцией Rust 2018. Некоторые другие линты, которые вы, возможно, захотите взять в привычку включать при старте нового проекта, — это `missing_docs` и `missing_debug_implementations`, которые предупреждают вас, если вы забыли задокументировать какие-либо публичные элементы в вашем крейте или добавить реализации `Debug` для каких-либо публичных типов соответственно.

Генерация тестов (Test Generation)

Написание хорошего набора тестов — это большой объём работы. И даже когда вы выполняете эту работу, тесты проверяют только тот конкретный набор поведений, которые вы рассматривали в момент их написания. К счастью, вы можете воспользоваться рядом техник генерации тестов, чтобы разрабатывать лучшие и более тщательные тесты. Они генерируют входные данные, которые вы используете для проверки корректности вашего приложения. Существует много таких инструментов, у каждого свои сильные и слабые стороны, так что здесь я освещу только основные стратегии, используемые этими инструментами: фаззинг (fuzzing) и тестирование на основе свойств (property testing).

Фаззинг (Fuzzing)

О фаззинге написаны целые книги, но на высоком уровне идея проста: генерировать случайные входные данные для вашей программы и смотреть, не падает ли она. Если программа падает — это баг. Например, если вы пишете библиотеку разбора URL, вы можете подвергнуть свою программу фазз-тестированию, систематически генерируя случайные строки и забрасывая

ими функцию разбора, пока она не запаникует. Сделанное наивно, это заняло бы некоторое время, чтобы дать результаты: если фаззер начинает с `a`, потом `b`, потом `c` и так далее, ему понадобится много времени, чтобы сгенерировать хитрый URL вроде `http://[:]`. На практике современные фаззеры используют метрики покрытия кода для исследования разных путей в вашем коде, что позволяет им достигать более высоких степеней покрытия быстрее, чем если бы входные данные действительно выбирались случайно.

Фаззеры отлично находят странные граничные случаи, которые ваш код обрабатывает некорректно. Они требуют небольшой настройки с вашей стороны: вы просто указываете фаззеру функцию, которая принимает «фаззируемый» вход, и поехали. Например, в листинге 7-7 показан пример того, как можно фаззировать парсер URL.

```
libfuzzer_sys::fuzz_target!(|data: &[u8]| {  
    if let Ok(s) = std::str::from_utf8(data) {  
        let _ = url::Url::parse(s);  
    }  
});
```

Листинг 7-7: Фаззинг парсера URL с помощью libfuzzer

Фаззер будет генерировать полуслучайные входные данные для замыкания, и любые из них, что образуют валидные UTF-8-строки, будут переданы парсеру. Обратите внимание, что код здесь не проверяет, успешен разбор или нет, — вместо этого он ищет случаи, когда парсер паникует или иным образом падает из-за нарушения внутренних инвариантов.

Фаззер продолжает работать, пока вы его не остановите, так что большинство инструментов фаззинга поставляются со встроенным механизмом остановки после того, как было исследовано определённое количество тестовых случаев. Если ваш вход не является тривиально фаззируемым типом — чем-то вроде хеш-таблицы, — вы обычно можете использовать крейт вроде `arbitrary`, чтобы превратить байтовую строку, которую генерирует фаззер, в более сложный тип `Rust`. Это кажется магией, но под капотом реализовано очень прямолинейно. Крейт определяет трейт `Arbitrary` с единственным методом `arbitrary`, который конструирует реализующий тип из источника случайных байтов. Примитивные типы вроде `u32` или `bool` читают необходимое количество байтов из этого входа, чтобы сконструировать валидный экземпляр самих себя, тогда как более сложные типы вроде `HashMap` или `BTreeSet` производят одно

число из входа, чтобы определить свою длину, а затем вызывают `Arbitrary` это число раз на своих внутренних типах. Есть даже атрибут `#[derive(Arbitrary)]`, который реализует `Arbitrary`, просто вызывая `arbitrary` на каждом содержащемся типе! Чтобы исследовать фаззинг дальше, я рекомендую начать с `cargo-fuzz`.

Тестирование на основе свойств (Property-Based Testing)

Иногда вы хотите проверить не только то, что ваша программа не падает, но и то, что она делает то, что от неё ожидается. Здорово, что ваша функция `add` не запаниковала, но если она сообщает, что результат `add(1, 4)` равен 68, то она, вероятно, всё равно неправильна. Вот здесь в игру вступает *тестирование на основе свойств (property-based testing)*; вы описываете ряд свойств, которым должен соответствовать ваш код, а затем фреймворк тестирования на основе свойств генерирует входные данные и проверяет, что эти свойства действительно выполняются.

Распространённый способ использовать тестирование на основе свойств — сначала написать простую, но наивную версию кода, который вы хотите протестировать, в корректности которой вы уверены. Затем для данного входа вы подаёте этот вход и в код, который хотите протестировать, и в упрощённую наивную версию. Если результат или вывод двух реализаций одинаков — ваш код хорош, это и есть свойство корректности, которое вы ищете, — а если нет, вы, вероятно, нашли баг. Вы также можете использовать тестирование на основе свойств для проверки свойств, не связанных напрямую с корректностью, например выполняются ли операции строго быстрее для одной реализации, чем для другой. Общий принцип в том, что вы хотите, чтобы любое различие в исходе между реальной и тестовой версиями было информативным и пригодным для действий, чтобы каждая неудача позволяла вам вносить улучшения. Наивной реализацией может быть та, что из стандартной библиотеки, которую вы пытаетесь заменить или дополнить (вроде `std::collections::VecDeque`), или это может быть более простая версия алгоритма, который вы пытаетесь оптимизировать (вроде наивного против оптимизированного умножения матриц).

Если этот подход генерации входных данных, пока не выполнится некоторое условие, звучит очень похоже на фаззинг, то это потому, что так и есть, — люди умнее меня доказывали, что фаззинг — это «просто» тестирование на

основе свойств, где проверяемое свойство — это «оно не падает».

Один недостаток тестирования на основе свойств в том, что оно сильнее опирается на предоставленные описания входных данных. Тогда как фаззинг будет продолжать пробовать все возможные входы, тестирование свойств обычно направляется аннотациями разработчика вроде «число между 0 и 64» или «строка, содержащая три запятые». Это позволяет тестированию свойств быстрее достигать случаев, на которые фаззеры могут долго наткаться случайным образом, но оно требует ручной работы и может упускать важные, но редкие багованные входы. По мере того как фаззеры и тестировщики свойств сближаются, однако, фаззеры начинают приобретать такого рода возможность поиска на основе ограничений.

Если вам интересна генерация тестов на основе свойств, я рекомендую начать с крейта `proptest`.

ТЕСТИРОВАНИЕ ПОСЛЕДОВАТЕЛЬНОСТЕЙ ОПЕРАЦИЙ (TESTING SEQUENCES OF OPERATIONS) Поскольку фаззеры и тестировщики свойств позволяют генерировать произвольные типы Rust, вы не ограничены тестированием одного вызова функции в вашем крейте. Например, скажем, вы хотите проверить, что некоторый тип `Foo` ведёт себя корректно, если вы выполняете на нём определённую последовательность операций. Вы могли бы определить перечисление `Operation`, которое перечисляет операции, и сделать так, чтобы ваша тестовая функция принимала `Vec<Operation>`. Затем вы могли бы инстанцировать `Foo` и выполнить каждую операцию на этом `Foo`, одну за другой. Большинство тестировщиков поддерживают минимизацию входных данных, так что они даже будут искать наименьшую последовательность операций, которая всё ещё нарушает свойство, если найден нарушающий свойство вход!

Дополнение тестов (Test Augmentation)

Скажем, у вас есть великолепный набор тестов, всё настроено, и ваш код проходит все тесты. Это прекрасно. Но затем, однажды, один из обычно надёжных тестов необъяснимо проваливается или падает с ошибкой сегментации (`segmentation fault`). Есть две распространённые причины такого рода недетерминированных падений тестов: состояния гонки (`race conditions`), при которых ваш тест может провалиться только если две операции

происходят в разных потоках в определённом порядке, и неопределённое поведение (*undefined behavior*) в небезопасном коде, например если какой-то небезопасный код читает определённое значение из неинициализированной памяти.

Ловить такого рода баги обычными тестами может быть трудно — часто у вас недостаточно низкоуровневого контроля над планированием потоков, раскладкой и содержимым памяти или другими более-менее случайными системными факторами, чтобы написать надёжный тест. Вы могли бы запускать каждый тест много раз в цикле, но это может не поймать ошибку, если плохой случай достаточно редок или маловероятен. К счастью, есть инструменты, которые могут помочь дополнить ваши тесты, чтобы ловить такого рода баги стало намного проще.

Первый из них — это потрясающий инструмент *Miri*, интерпретатор внутреннего среднеуровневого представления Rust (*mid-level intermediate representation, MIR*). MIR — это внутреннее, упрощённое представление Rust, которое помогает компилятору находить оптимизации и проверять свойства, не имея дела со всем синтаксическим сахаром самого Rust. Прогон ваших тестов через Miri так же прост, как запуск `cargo miri test`. Miri *интерпретирует* ваш код, а не компилирует и запускает его, как обычный бинарный файл, что делает тесты значительно медленнее. Но взамен Miri может отслеживать всё состояние программы по мере выполнения каждой строки вашего кода. Это позволяет Miri обнаруживать и сообщать, если ваша программа когда-либо проявляет определённые типы неопределённого поведения, такие как чтение неинициализированной памяти, использование значений после того, как они были сброшены (*dropped*), или обращения по указателю за пределами границ. Вместо того чтобы эти операции вели к странному поведению программы, которое может лишь иногда приводить к наблюдаемым провалам тестов (вроде падений), Miri обнаруживает их в момент возникновения и немедленно сообщает вам.

Например, рассмотрим совершенно небезопасный код в листинге 7-8, который создаёт две эксклюзивные ссылки на значение.

```
let mut x = 42;
let x: *mut i32 = &mut x;
let (x1, x2) = unsafe { (&mut *x, &mut *x) };
println!("{}", x1, x2);
```

Листинг 7-8: Дико небезопасный код, некорректность которого обнаруживает Miri

На момент написания, если вы прогоните этот код через Miri, вы получите ошибку, которая точно укажет, что не так:

```
error: Undefined Behavior: trying to reborrow for Unique at alloc1383, but
parent tag <2772> does not have an appropriate item in the borrow stack
--> src/main.rs:4:6
  |
4 | let (x1, x2) = unsafe { (&mut *x, &mut *x) };
  |           ^^ trying to reborrow for Unique at alloc1383, but parent tag <2772>
does not have an appropriate item in the borrow stack
```

ПРИМЕЧАНИЕ Miri всё ещё находится в разработке, и его сообщения об ошибках не всегда самые лёгкие для понимания. Это проблема, над которой активно работают, так что к тому времени, когда вы это читаете, вывод ошибок, возможно, уже стал намного лучше!

Ещё один инструмент, на который стоит обратить внимание, — это *Loom*, хитроумная библиотека, которая пытается обеспечить, чтобы ваши тесты запускались с каждым релевантным чередованием конкурентных операций. На высоком уровне Loom отслеживает все точки синхронизации между потоками и запускает ваши тесты снова и снова, каждый раз корректируя порядок, в котором потоки проходят эти точки синхронизации. Так что если поток A и поток B оба захватывают один и тот же `Mutex`, Loom обеспечит, чтобы тест запустился один раз с A, захватывающим его первым, и один раз с B, захватывающим его первым. Loom также отслеживает атомарные обращения, упорядочивания памяти (memory orderings) и обращения к `UnsafeCell` (который мы обсудим в главе 10) и проверяет, что потоки не обращаются к ним неподобающим образом. Если тест проваливается, Loom может дать вам точную сводку того, в каком порядке выполнялись потоки, чтобы вы могли определить, как произошло падение.

Тестирование производительности (Performance Testing)

Писать тесты производительности трудно, потому что часто сложно точно смоделировать реальную нагрузку, отражающую использование вашего крейта в реальном мире. Но иметь такие тесты важно; если ваш код вдруг работает в 100 раз медленнее, это действительно должно считаться багом,

однако без теста производительности вы можете не заметить регрессию. Если ваш код работает в 100 раз *быстрее*, это тоже может указывать на то, что что-то не так. Оба этих момента — хорошие причины иметь автоматизированные тесты производительности как часть вашего CI: если производительность резко меняется в любом направлении, вам следует об этом знать.

В отличие от функционального тестирования, тесты производительности не имеют общего, чётко определённого вывода. Функциональный тест либо проходит, либо проваливается, тогда как тест производительности может дать вам число пропускной способности, профиль задержки, число обработанных образцов или любую другую метрику, которая может быть релевантна рассматриваемому приложению. Кроме того, тест производительности может потребовать запуска функции в цикле несколько сотен тысяч раз или может занимать часы, работая на распределённой сети многоядерных машин. По этой причине трудно говорить о том, как писать тесты производительности в общем смысле. Вместо этого в этом разделе мы рассмотрим некоторые из проблем, с которыми вы можете столкнуться при написании тестов производительности в Rust, и как их смягчить. Три особенно распространённые ловушки, которые часто упускают из виду, — это вариативность производительности, оптимизации компилятора и накладные расходы ввода-вывода. Давайте исследуем каждую из них по очереди.

Вариативность производительности (Performance Variance)

Производительность может варьироваться по огромному множеству причин, и множество факторов влияет на то, насколько быстро выполняется конкретная последовательность машинных инструкций. Некоторые очевидны, такие как тактовая частота процессора и памяти или насколько машина загружена в остальном, но многие гораздо более тонки. Например, ваша версия ядра может изменить производительность страничной подкачки (paging), длина вашего имени пользователя может изменить раскладку памяти, а температура в комнате может вызвать снижение тактовой частоты процессора. В конечном счёте, крайне маловероятно, что если вы запустите бенчмарк дважды, вы получите одинаковый результат. На самом деле, вы можете наблюдать существенную вариативность, даже если используете одно и то же оборудование. Или, рассматривая с другой стороны, ваш код мог стать медленнее или быстрее, но эффект может быть невидим из-за различий в среде бенчмаркинга.

Не существует идеальных способов устранить всю вариативность в ваших результатах производительности, если только вам не посчастливится запускать бенчмарки многократно на крайне разнообразном парке машин. Тем не менее важно постараться обработать эту вариативность измерений как можно лучше, чтобы извлечь сигнал из шумных измерений, которые дают нам бенчмарки. На практике наш лучший друг в борьбе с вариативностью — запускать каждый бенчмарк много раз, а затем смотреть на *распределение (distribution)* измерений, а не просто на одно. У Rust есть инструменты, которые могут помочь с этим. Например, вместо вопроса «Сколько времени в среднем заняло выполнение этой функции?» крейты вроде `hdrhistogram` позволяют нам смотреть на статистику вроде «Какой диапазон времени выполнения покрывает 95% наблюдаемых нами образцов?». Чтобы быть ещё строже, мы можем использовать техники вроде проверки нулевой гипотезы (null hypothesis testing) из статистики, чтобы укрепить некоторую уверенность в том, что измеренное различие действительно соответствует реальному изменению, а не просто является шумом.

Лекция о статистической проверке гипотез выходит за рамки этой книги, но, к счастью, большая часть этой работы уже была проделана другими. Крейт `criterion`, например, делает всё это и даже больше за вас. Всё, что вам нужно, — это дать ему функцию, которую он может вызывать для запуска одной итерации вашего бенчмарка, и он запустит её соответствующее число раз, чтобы быть достаточно уверенным, что результат надёжен. Затем он создаёт отчёт по бенчмарку, который включает сводку результатов, анализ выбросов (outliers) и даже графические представления трендов во времени. Конечно, он не может устранить эффекты тестирования только на конкретной конфигурации оборудования, но он по крайней мере категоризирует шум, который измерим между запусками.

Оптимизации компилятора (Compiler Optimizations)

Компиляторы в наши дни действительно умны. Они устраняют мёртвый код, вычисляют сложные выражения во время компиляции, разворачивают циклы и выполняют другую тёмную магию, чтобы выжать каждую каплю производительности из нашего кода. Обычно это здорово, но когда мы пытаемся измерить, насколько быстр конкретный участок кода,сообразительность компилятора может дать нам некорректные результаты. Например, возьмём код для бенчмарка `Vec::push` в листинге 7-9.

```
let mut vs = Vec::with_capacity(4);
let start = std::time::Instant::now();
for i in 0..4 {
    vs.push(i);
}
println!("took {:?}", start.elapsed());
```

Листинг 7-9: Подозрительно быстрый бенчмарк производительности

Если бы вы взглянули на ассемблерный вывод этого кода, скомпилированного в режиме релиза с помощью чего-то вроде превосходного *godbolt.org* или *cargo-asm*, вы бы сразу заметили, что что-то не так: вызовов `Vec::with_capacity` и `Vec::push`, да и всего цикла `for`, нигде не видно. Они были полностью оптимизированы. Компилятор понял, что ничему в коде на самом деле не требуются операции с вектором, и устранил их как мёртвый код. Конечно, компилятор полностью в своём праве так поступить, но для целей бенчмаркинга это не особенно полезно.

Чтобы избежать такого рода оптимизаций при бенчмаркинге, стандартная библиотека предоставляет `std::hint::black_box`. Эта функция была темой множества дебатов и путаницы и на момент написания всё ещё ожидает стабилизации, но она настолько полезна, что её всё же стоит здесь обсудить. По своей сути это просто функция тождества (которая принимает `x` и возвращает `x`), которая сообщает компилятору считать, что аргумент функции используется произвольными (легальными) способами. Она не препятствует тому, чтобы компилятор применял оптимизации ко входному аргументу, и не препятствует тому, чтобы компилятор оптимизировал то, как используется возвращаемое значение. Вместо этого она побуждает компилятор действительно вычислить аргумент (в предположении, что он будет использован) и сохранить этот результат где-то, доступном процессору, так чтобы `black_box` мог быть вызван с вычисленным значением. Компилятор волен, скажем, вычислить входной аргумент во время компиляции, но он всё равно должен внедрить результат в программу.

Этой функции достаточно для многих, хотя и, признаться, не для всех, наших нужд бенчмаркинга. Например, мы можем аннотировать листинг 7-9 так, чтобы обращения к вектору больше не оптимизировались, как показано в листинге 7-10.

```

let mut vs = Vec::with_capacity(4);
let start = std::time::Instant::now();
for i in 0..4 {
    black_box(vs.as_ptr());
    vs.push(i);
    black_box(vs.as_ptr());
}
println!("took {:?}", start.elapsed());

```

Листинг 7-10: Исправленная версия листинга 7-9

Мы сказали компилятору считать, что `vs` используется произвольными способами на каждой итерации цикла, как до, так и после вызовов `push`. Это вынуждает компилятор выполнять каждый `push` по порядку, не сливая воедино и не оптимизируя иным образом последовательные вызовы, поскольку он должен считать, что «произвольные вещи, которые нельзя оптимизировать» (это и есть часть `black_box`) могут произойти с `vs` между каждым вызовом.

Обратите внимание, что мы использовали `vs.as_ptr()`, а не, скажем, `&vs`. Это из-за оговорки, что компилятор должен считать, что `black_box` может выполнить любую *легальную* операцию над своим аргументом. Изменять `Vec` через разделяемую ссылку нелегально, так что если бы мы использовали `black_box(&vs)`, компилятор мог бы заметить, что `vs` не изменится между итерациями цикла, и реализовать оптимизации на основе этого наблюдения!

Измерение накладных расходов ввода-вывода (I/O Overhead Measurement)

При написании бенчмарков легко случайно измерить не то, что нужно. Например, нам часто хочется получать информацию в реальном времени о том, насколько далеко продвинулся бенчмарк. Чтобы это сделать, мы могли бы написать код вроде того, что в листинге 7-11, предназначенный для измерения того, насколько быстро выполняется `my_function`:

```

let start = std::time::Instant::now();
for i in 0..1_000_000 {
    println!("iteration {}", i);
    my_function();
}
println!("took {:?}", start.elapsed());

```

Листинг 7-11: Что мы на самом деле здесь измеряем?

Это может выглядеть так, будто оно достигает цели, но в реальности оно не измеряет то, насколько быстро выполняется `my_function`. Вместо этого, скорее всего, этот цикл, скорее всего, скажет нам, сколько времени занимает печать миллиона чисел. `println!` в теле цикла выполняет много работы за кулисами: он превращает двоичное целое число в десятичные цифры для печати, блокирует стандартный вывод, записывает последовательность кодовых точек UTF-8, используя по меньшей мере один системный вызов, а затем снимает блокировку стандартного вывода. Мало того, системный вызов может ещё и заблокироваться, если ваш терминал медленно печатает получаемый им ввод. Это очень много тактов! А время, которое требуется на вызов `my_function`, может на этом фоне выглядеть бледно.

Похожее происходит, когда ваш бенчмарк использует случайные числа. Если вы запустите `my_function(rand::random())` в цикле, вы вполне можете в основном измерять время, которое требуется на генерацию миллиона случайных чисел. То же касается получения текущего времени, чтения файла конфигурации или запуска нового потока — все эти вещи занимают много времени, относительно говоря, и могут в итоге затмить время, которое вы на самом деле хотели измерить.

К счастью, эту конкретную проблему часто легко обойти, как только вы о ней знаете. Убедитесь, что тело вашего цикла бенчмаркинга содержит почти ничего, кроме конкретного кода, который вы хотите измерить. Весь остальной код должен выполняться либо до начала бенчмарка, либо вне измеряемой части бенчмарка. Если вы используете `criterion`, взгляните на различные таймирующие циклы, которые он предоставляет, — все они там, чтобы обслуживать случаи бенчмаркинга, требующие разных стратегий измерения!

Итоги (Summary)

В этой главе мы исследовали встроенные возможности тестирования, которые Rust предлагает в большом объёме. Мы также рассмотрели ряд средств и техник тестирования, полезных при тестировании кода на Rust. Это последняя глава, которая фокусируется на более высокоуровневых аспектах промежуточного использования Rust в этой книге. Начиная со следующей главы, посвящённой декларативным и процедурным макросам, мы будем гораздо больше сосредотачиваться на самом коде Rust. Увидимся на следующей странице!